

Vectors and Embeddings: Meaning as Coordinates

FRIDAY AI Education — Episode 005 · Printable Companion Paper

Abstract

This paper explains how machines turn meaning into numbers. The central idea is the **embedding**: a way of representing a word, sentence, document, or image as a list of coordinates in a high-dimensional space, arranged so that things with similar meaning land near each other. Once meaning has coordinates, "find me something related to this" becomes a geometry problem — measure distance, return the nearest points. That single move is what powers semantic search, recommendation, retrieval for AI assistants, and the searchable knowledge bases that FRIDAY and GBrain are built on.

The goal here is not to make you an ML engineer. It is to give you a correct mental model: what an embedding actually is, why nearness in this space is useful, where it quietly fails, and what that means when you are deciding which workflows to trust to a machine. The most important sentence in the paper is also the least exciting: **embeddings represent similarity, not truth**. Hold onto that and most of the rest follows.

1. Why this matters before we define anything

Most of the practical value a founder will get from AI in the next few years is not the model writing poetry. It is the boring, compounding advantage of being able to ask your own information a question in plain language and get a precise answer. "What did we decide about pricing in the March board call?" "Find every podcast clip where this person talks about hiring." "Which of our contracts have a most-favored-nation clause?"

Keyword search cannot do this reliably. The word "pricing" might never appear; the speaker said "what we charge." Keyword search matches *strings*. What you actually want is to match *meaning*. Embeddings are the mechanism that makes matching-by-meaning possible, and a **vector database** is the place you keep those meanings so you can search them fast.

If you understand this one concept well, a large category of "can AI do X for my company?" questions resolves into a clear yes, no, or yes-but. That is worth a morning walk.

2. The core idea: meaning as coordinates

Start with a deliberately simple picture.

Imagine you wanted to describe animals using just two numbers: one for *size* and one for *domestication*. A house cat is small and domestic. A wolf is medium and wild. A cow is large and domestic. Each animal becomes a pair of coordinates — a point on a 2D map. Animals that are alike in those two respects sit near each other. A dog lands close to the cat; the wolf sits off near the dog but pulled toward "wild."

You have just built a tiny **embedding**: a representation of things as points in a space, where position encodes properties, and *nearness encodes similarity*.

Now scale the idea up in two ways.

First, stop hand-picking the dimensions. Instead of you deciding "size" and "domestication," a model *learns* which dimensions matter by looking at enormous amounts of data. Nobody labels what each dimension means, and mostly they don't correspond to a clean human word. They are just directions in the space that turned out to be useful for telling things apart.

Second, use far more than two dimensions. Real text embeddings have hundreds or thousands of them — a modern model might give you a list of 1,536 numbers per chunk of text. You cannot visualize 1,536 dimensions, and you don't need to. The 2D animal map is a faithful intuition; the only thing that changes is the number of coordinates.

That list of numbers is a **vector**. An embedding *is* a vector — the term "embedding" emphasizes *what it's for* (representing meaning), and "vector" emphasizes *what it is* (an ordered list of numbers). When someone says "embed this document," they mean "run it through a model and get back the vector that represents it."

The famous, slightly magical demonstration of this — worth seeing once — is that the learned directions can encode relationships. In a well-trained word embedding, the vector for *king*, minus *man*, plus *woman*, lands near *queen*. The model was never told about gender or royalty. The geometry fell out of usage. That is the headline result from the Word2vec work in the reading list, and it is the cleanest evidence that these coordinates are capturing something real about meaning rather than just shuffling strings.

A note to keep your feet on the ground: this does not mean the model *understands* king and queen the way you do. It means that across billions of sentences, those words were used in patterns regular enough that geometry could capture the pattern. Useful, not mystical.

3. The mechanism underneath

Where do the numbers come from? The honest short answer: a neural network trained on a simple, repetitive task learns them as a side effect.

The classic version (Word2vec) works like this. Take a huge pile of text. Slide a window across it and play a fill-in-the-blank game: given the surrounding words, predict the missing one — or given a word, predict its neighbors. The model starts with random vectors for every word and adjusts them, slightly, every time it guesses, nudging the numbers toward configurations that make good predictions.

The engine behind this is an old, reliable idea, usually credited to the linguist J.R. Firth: "*You shall know a word by the company it keeps.*" Words that appear in similar contexts — "doctor" and "physician," "Tuesday" and "Thursday" — get pushed toward similar coordinates, because the model can predict their contexts in similar ways. Meaning, in this approach, is *distributional*: it is inferred from patterns of co-occurrence, not from any definition.

Modern embedding models are more sophisticated than Word2vec in two ways that matter to you:

1. **They embed whole passages, not just single words.** Word2vec gives one vector per word. The models you'll actually use (such as OpenAI's embedding models) take a chunk of text — a sentence, a paragraph, a clause — and return a single vector representing the meaning of the *whole thing*, with word order and context taken into account. "The bank raised rates" and "the river bank eroded" produce different vectors for "bank," because the surrounding words are part of the input.

2. **They are trained to make *similar meanings* sit close**, often by showing the model pairs of texts that mean the same thing and pairs that don't, and tuning the space so the matching pairs are near and the others are far.

The output is always the same shape: text in, a fixed-length vector out. The same model, applied consistently, is what makes two pieces of text comparable. (Important operational detail: vectors from *different* models are not comparable. You cannot mix embeddings from one model with another and expect distances to mean anything. Pick one model for a given index and stick with it.)

You do not need to know how the network computes this any more than you need to know how a database B-tree works to use an index. You need to know what goes in, what comes out, and what the output is good for.

4. Semantic similarity and nearest-neighbor search

Once everything you care about is a vector, "what is related to this?" becomes "what is *near* this?"

Semantic similarity is just a measurement of how close two vectors are. The standard measure is the *angle* between them (cosine similarity) — roughly, are these two arrows pointing in the same direction? Two passages about layoffs point the same way even if one says "reduction in force" and the other says "we let people go." A passage about layoffs and one about pizza toppings point in wildly different directions. The number you get back, often between 0 and 1, is a similarity score.

One discipline to internalize now: **call it semantic similarity or vector distance — never geographic closeness**. "Near" here is about meaning, not space or time. Two documents written years apart on different continents can be the nearest neighbors in this space because they say the same thing. The geometry is a metaphor for meaning, and the metaphor breaks if you start thinking of it as a literal map.

Nearest-neighbor search is the operation that uses this. You take a query, embed it into the same space, and ask: which stored vectors are closest? Return them, ranked by similarity. That is the entire retrieval mechanism behind semantic search.

The catch is scale. If you have ten thousand documents, you could compare your query to every single one and pick the closest — that's *exact* nearest-neighbor search, and it's fine. If you have fifty million, comparing against all of them on every query is too slow. So in practice systems use **approximate nearest-neighbor (ANN)** search: clever index structures that find *almost* the closest matches while skipping most of the comparisons. You trade a sliver of accuracy for a massive speedup. For nearly every business use case this trade is invisible and correct — but know that it exists, because "approximate" is one of the quiet places error enters.

5. Chunking: the unglamorous decision that determines quality

Here is the part teams underestimate, so it gets its own section.

You rarely embed a whole document as one vector. A forty-page contract reduced to a single point in space is a smear — it's "about" everything and therefore retrieves well for nothing. So you split documents into pieces first. That splitting is called **chunking**, and how you do it has more effect on real-world search quality than almost any other choice, including which model you pick.

Chunk too large, and each vector is a blurry average of several distinct ideas; relevant passages get buried inside irrelevant context. Chunk too small, and you shatter meaning across fragments — a sentence that only makes sense

with the one before it gets stranded, and the retrieval returns a true-but-useless snippet.

The art is to chunk along the *natural seams of meaning*: by paragraph, by section, by a single speaker's turn in a transcript, by a logical clause in a contract. A good chunk is a self-contained unit of meaning — something that, read on its own, still answers a question. Overlapping the chunks slightly (so the end of one repeats at the start of the next) is a common trick to avoid cutting an idea in half at the boundary.

Concrete version for the podcast-clip example: if you want to "find every clip where this person talks about hiring," you chunk the transcript into coherent segments — ideally aligned to speaker turns or topic shifts, each tagged with its timestamp — embed each segment, and search. Chunk by fixed 30-second blocks instead, and you'll routinely retrieve a clip that *starts* mid-thought or *ends* right before the good part. Same model, same database, dramatically worse product, entirely because of chunking.

Remember this when you evaluate a vendor or a build: if semantic search is "not finding things," the first suspect is almost never the model. It is the chunking.

6. The vector database: where meaning gets stored and searched

A **vector database** is a system built to store large numbers of vectors and answer nearest-neighbor queries over them quickly. That's its defining job. Alongside each vector it keeps **metadata** — the original text, the source document, author, date, timestamps, permissions — so that when a vector is retrieved you can show the human-readable thing it came from, and so you can *filter* (e.g., "nearest neighbors, but only from documents this user is allowed to see, only from the last year").

That filtering point is not a footnote — it is where governance lives. A vector store that ignores permissions will happily retrieve a passage from a document the asker should never see. In an organization, "who can retrieve what" is a real access-control problem wearing a search-engine costume.

The pipeline, end to end, is small enough to hold in your head:

Stage	What happens	Where it can go wrong
Ingest	Collect source material (docs, transcripts, pages)	Missing or stale sources; nothing to find if it's not in here
Chunk	Split into self-contained units of meaning	Chunks too big (blurry) or too small (fragmented)
Embed	Run each chunk through the model → a vector	Wrong/mismatched model; vectors not comparable across models
Store	Save vectors + text + metadata in the vector DB	No permissions or filters; can't trace results to sources
Query	Embed the question, find nearest neighbors	Approximation error; query phrased unlike the stored text
Return	Rank by similarity, hand back text + citations	Confident-looking matches that are merely <i>similar</i> , not <i>right</i>

GBrain, in FRIDAY's architecture, is the substrate that owns this pipeline: it is the memory and knowledge layer where the organization's material is ingested, chunked, embedded, stored, and made retrievable. When FRIDAY needs to answer something or take an action grounded in what the organization knows, it queries GBrain rather than relying on the model's general training. The model supplies reasoning and language; GBrain supplies the *specific, current, sourced facts*. Keeping those two jobs separate is the whole point — and it's why the search layer matters as

much as the model.

7. Where this connects: retrieval as the bridge between a model and your reality

A large language model knows a great deal in general and *nothing specific* about your company. It wasn't trained on your contracts, your call transcripts, or last week's decisions. Worse, asked a question it can't ground, it will often produce a fluent, confident, wrong answer.

The standard fix is **RAG — retrieval augmented generation** (say it in full at least once; it is not "RAIG"). The shape is simple: before the model answers, you *retrieve* the relevant chunks from your vector database and feed them to the model along with the question. The model then answers *from the supplied passages* and can cite them. Embeddings and nearest-neighbor search are the "retrieval" half. RAG is the dominant pattern for making a general model useful on private, current, specific information — and it sits directly on everything in this paper.

This is also where your first practical wedge lives. Making a person's or company's public body of work — every talk, post, interview, and clip — searchable, reusable, and *citable* by meaning is a RAG system end to end: ingest the public material, chunk it sensibly, embed it, store it with sources, and let people query it in plain language and get answers that point back to the original. The entire value proposition rests on the quality of the retrieval, which rests on the six concepts in this paper.

And it is where the boundary lives, which is the next section.

8. The limitation: similar is not the same as correct

This is the section to actually annotate.

Embeddings measure **similarity**, and similarity is a genuinely different thing from **truth** or **relevance-to-your-intent**. The space will faithfully tell you which passages are *most alike in meaning* to your query. It has no idea which one is *correct, current, authoritative, or what you actually meant*. Several specific failure modes follow from this:

Confidently similar, actually wrong. A query about your *current* refund policy may retrieve a passage about the 2021 refund policy, because the two are nearly identical in meaning — that's exactly why it scored high. Similarity rewards resemblance, and an outdated version of a thing resembles the current version more than almost anything else. Recency, authority, and correctness are *not* in the geometry unless you put them there with metadata and filtering.

Plausible neighbors to a false premise. Ask a leading or wrong question and the system cheerfully finds passages similar to your wrong framing. The retrieval looks successful — results came back, scores were high — while quietly confirming a mistake.

The gap between similar and intended. "Find clips about our biggest risk" might surface every passage where someone says the word "risk," while the clip you wanted is one where the CEO describes a looming customer concentration problem without ever using the word. Embeddings narrow the gap between keyword and meaning enormously, but they do not read your mind.

Garbage in, fluent garbage out. If the source material is wrong, stale, or missing, retrieval will faithfully return the wrong, stale thing — or nothing — and a generation model will wrap it in confident prose. The system has no built-in

sense of whether its sources deserve trust.

The operating conclusion: **embeddings make information findable, not trustworthy**. They are a superb first-stage filter — narrowing a million documents to the ten worth looking at — and a poor final authority. The way you get trustworthiness back is not better geometry. It is the surrounding machinery: metadata and filters (recency, source, permissions), **citations** so a human can verify the actual source, **evals** (evaluations — measured tests of whether retrieval returns the right things, not "evils"), and **human approval gates** for anything consequential. Retrieval finds candidates; governance decides what to trust and what to act on.

This is the line a serious person will hold in a serious conversation: not "AI search is unreliable, avoid it" and not "we have semantic search, so it's handled," but "retrieval surfaces likely-relevant material with measured recall; trust and action are governed separately." That is the accurate, defensible position.

9. Choosing where to apply this

A useful frame for deciding whether a workflow is a good early candidate: **embeddings are safest where being approximately right and fully traceable is enough, and a human makes the final call**.

Strong early candidates share three traits — a human reviews the output, every result links back to a verifiable source, and an occasional miss is cheap. Searching company docs to *assist* a person drafting an answer fits perfectly: the human is in the loop, sources are cited, a wrong retrieval is caught and discarded at no cost. Surfacing relevant podcast clips for a producer to review, same thing.

Weak candidates invert those traits: no human review, automated action on the result, and a costly or irreversible miss. "Automatically email customers the policy our system retrieved" is the kind of thing that turns a chunking bug into an incident. The concept didn't fail; it was deployed past its boundary.

So the test for any proposed workflow is short: *Is a human approving the consequential step? Can every answer be traced to a source? Is a wrong retrieval cheap to catch?* Three yeses means embeddings are doing what they're good at. A no is not a veto — it's a sign you need more governance around the retrieval before you let it act.

10. Vocabulary

Embedding — A representation of a piece of content (word, sentence, document, image) as a vector of numbers, arranged so that similar meanings produce nearby vectors. The thing you store and search.

Vector — An ordered list of numbers; coordinates of a point in a high-dimensional space. An embedding *is* a vector; the two words emphasize purpose vs. form.

Dimension — One coordinate axis in the space. Modern text embeddings have hundreds to thousands. Individual dimensions usually don't map to a clean human concept.

Semantic similarity — How close two vectors are in meaning, measured by distance or (usually) the angle between them. Always "semantic" — about meaning — never geographic or temporal closeness.

Nearest-neighbor search — Given a query vector, find the stored vectors closest to it. The core retrieval operation.

Approximate nearest-neighbor (ANN) — A faster version that finds *almost* the closest matches by skipping most comparisons; trades a little accuracy for large speed gains at scale.

Vector database — A system for storing many vectors with their metadata and answering nearest-neighbor queries quickly, including filtering by permissions, recency, and source.

Chunking — Splitting documents into self-contained units of meaning before embedding. The most underrated determinant of search quality.

Metadata — The non-vector information stored alongside each vector (source, date, author, permissions, timestamps) used for display, citation, and filtering.

RAG (retrieval augmented generation) — Retrieving relevant chunks and feeding them to a language model so it answers from your specific, current material and can cite sources. The dominant pattern for grounding a general model in private data.

Evals — Evaluations: measured tests of whether the system returns the right results. The discipline that turns "feels like it works" into "we know it works." (Not "evils.")

11. Reading guide

Two sources, read in this order. Together they cover roughly two focused hours — not a course, just the two anchors that repay the time.

1. Jay Alamar — *The Illustrated Word2vec* · <https://jalammar.github.io/illustrated-word2vec/>

Read this first, for *intuition*. It builds the picture this paper used — meaning as coordinates, the king–man+woman result, learning vectors from context — with clear visuals and no equations to wade through. *Read for*: why nearby vectors mean similar things and how the numbers get learned in the first place. *Skim*: the deeper training-mechanics passages (negative sampling, etc.) — worth seeing once, not worth memorizing. As you read, keep asking "what would the *space* look like here?" — that habit is the whole skill.

2. OpenAI — *Embeddings guide* · <https://platform.openai.com/docs/guides/embeddings>

Read this second, for the *practical reality*. It shows what you actually get back from a current embedding model (a fixed-length vector), and names the real use cases — search, clustering, recommendation, classification. *Read for*: the concrete shape of the input/output and the catalog of what embeddings are used for in production. *Note especially*: that embeddings from different models aren't interchangeable, and that similarity scores are relative, not absolute truth — the operational version of this paper's main warning.

If you only have time for one: read Alamar for the *why*, and let the OpenAI guide wait until you're sizing a real build.

12. Walking questions

Carry these on the next walk. They're ordered to move from "do I get it?" to "where do I use it?"

1. Could you, right now, explain to a sharp non-technical colleague why "we let people go" and "reduction in force" land near each other in embedding space — without using jargon?

2. Which part still feels blurry: how the numbers are *learned*, why *nearness* equals *similarity*, or where the *approximation* in fast search creeps in?
3. Where does this already show up in FRIDAY and GBrain — and which stage of the ingest → chunk → embed → store → query → return pipeline would you bet is the weakest link today?
4. Name one bounded workflow where you could test semantic search *safely* — human in the loop, every answer cited, a wrong result cheap to catch. What would make you call it a success, and how would you *measure* that rather than eyeball it?
5. Where would you be tempted to let retrieval *act* on its own — and what governance (filters, citations, evals, an approval gate) would have to exist first before that temptation became responsible?

One-sentence takeaway

Embeddings turn meaning into coordinates so that "find what's related" becomes "find what's near" — a powerful way to surface likely-relevant information, and never, on its own, a source of truth.